

TAREA 06 WORKFLOW GITFLOW

Gitflow

Estudiante:

Raul Ernesto Quintanilla Espinal

Actividad:

Practica el flujo de ramas de gitflow a mano

Fecha:

07 de Octubre de 2026

Introducción

El control de versiones constituye el pilar fundamental para el desarrollo de software colaborativo, ordenado y escalable en entornos profesionales modernos. A medida que los equipos de ingeniería crecen, la gestión de múltiples funcionalidades simultáneas, correcciones de errores urgentes y publicaciones a producción requiere una estrategia clara de ramificación (branching model).

Entre los diversos modelos existentes en la industria, Gitflow —propuesto originalmente por Vincent Driessen en 2010— se ha consolidado como un estándar robusto y altamente didáctico. Gitflow se estructura en torno a ramas con roles estrictos: dos ramas de vida infinita ('main' y 'develop') y ramas de soporte temporal ('feature/*', 'release/*', 'hotfix/*'). En esta práctica de laboratorio, se desarrolla de forma manual y secuencial el ciclo básico de Gitflow en la terminal, creando ramas de características, integrándolas en la rama de desarrollo mediante fusiones controladas y finalmente promocionando los cambios a la rama de producción, preparando las bases para futuras implementaciones con CI/CD.

Objetivo

Objetivo General:

Comprender, ejecutar y validar manualmente el modelo de flujo de trabajo Gitflow en un repositorio Git local y sincronizado con GitHub, simulando la creación, integración y puesta en producción de dos funcionalidades independientes.

Objetivos Específicos:

- Inicializar y configurar correctamente la rama de integración 'develop' a partir de la rama estable 'main'.
- Crear y trabajar de forma aislada en dos ramas de soporte de características: 'feature/login' y 'feature/registro'.
- Integrar cada rama de característica dentro de 'develop' empleando fusiones sin avance rápido (--no-ff) para mantener un historial trazable.
- Simular una liberación a producción fusionando la rama 'develop' consolidada hacia 'main'.
- Inspeccionar y documentar la topología del historial mediante el comando gráfico 'git log --graph --oneline --all'.

Marco Teórico: Estructura de Ramas en Gitflow

El modelo Gitflow establece una separación estricta entre el código en desarrollo continuo y el código listo para despliegue:

Rama	Ciclo de Vida	Propósito en Gitflow
main / master	Permanente	Contiene el código estable que se encuentra actualmente en producción. Cada commit refleja una versión desplegable.
develop	Permanente	Rama de integración continua. Agrupa todas las características terminadas y probadas antes de ser liberadas.
feature/*	Temporal	Nacen desde 'develop' y se fusionan de vuelta a 'develop'. Aíslan el desarrollo de nuevas pantallas, componentes o APIs.
release / hotfix	Temporal	Ramas para estabilización previa a producción (release) o parches de emergencia directos a producción (hotfix).

Guía de Ejecución Paso a Paso en Terminal

A continuación se presenta el procedimiento cronológico llevado a cabo en el computador, con los comandos ejecutados, su explicación técnica y las buenas prácticas aplicadas:

Paso 0: Preparación y Ubicación en el Repositorio

Abrimos la consola de comandos (Git Bash, Terminal o PowerShell) y navegamos hacia el directorio del proyecto de las sesiones prácticas anteriores:

```
$ cd ~/proyectos/practica-git
$ git status
```

Paso 1: Creación y verificación de la rama develop

Nos aseguramos de posicionarnos sobre la rama 'main' actualizada y, a partir de ella, creamos la rama 'develop' (si no existía previamente) para convertirla en el eje de integración:

```
$ git checkout main
$ git checkout -b develop
Switched to a new branch 'develop'
```

Paso 2: Creación de la rama feature/login y registro de commit

Estando en 'develop', ramificamos para dar soporte a la funcionalidad de inicio de sesión. Creamos el archivo login.txt con el contenido exacto requerido y registramos el commit:

```
$ git checkout develop
$ git checkout -b feature/login
$ echo "Formulario de login" > login.txt
$ git add login.txt
$ git commit -m "feat(login): Añade formulario de login"
[feature/login 2b1a0f9] feat(login): Añade formulario de login
1 file changed, 1 insertion(+)
create mode 100644 login.txt
```

Paso 3: Fusión (merge) de feature/login en develop

Una vez completada y testeada la funcionalidad, regresamos a 'develop' para incorporar los cambios. Se utiliza la bandera '--no-ff' (no fast-forward) para forzar la creación de un commit de fusión explícito, lo que garantiza que la existencia de la rama feature quede documentada en el historial:

```
$ git checkout develop
$ git merge --no-ff feature/login -m "Merge branch 'feature/login' into develop"
Merge made by the 'ort' strategy.
login.txt | 1 +
1 file changed, 1 insertion(+)
create mode 100644 login.txt
```

Paso 4: Creación de la rama feature/registro y registro de commit

Nuevamente partiendo desde la rama 'develop' (que ahora ya incluye la funcionalidad de login), creamos la rama para el formulario de registro y registramos el correspondiente archivo:

```
$ git checkout develop
$ git checkout -b feature/registro
$ echo "Formulario de registro" > registro.txt
$ git add registro.txt
$ git commit -m "feat(registro): Añade formulario de registro"
[feature/registro 5c4b3a2] feat(registro): Añade formulario de registro
1 file changed, 1 insertion(+)
create mode 100644 registro.txt
```

Paso 5: Fusión (merge) de feature/registro en develop

Regresamos a 'develop' y fusionamos la rama 'feature/registro', consolidando ambas características:

```
$ git checkout develop
$ git merge --no-ff feature/registro -m "Merge branch 'feature/registro' into develop"
Merge made by the 'ort' strategy.
registro.txt | 1 +
1 file changed, 1 insertion(+)
create mode 100644 registro.txt
```

Paso 6: Publicación / Despliegue a Producción (main)

Con todas las características integradas y estables en 'develop', procedemos a simular el pase a producción. Nos situamos en la rama 'main' y fusionamos 'develop':

```
$ git checkout main
$ git merge --no-ff develop -m "Release: Publicación de versión a producción con login y registro"
Merge made by the 'ort' strategy.
 login.txt | 1 +
 registro.txt | 1 +
 2 files changed, 2 insertions(+)
 create mode 100644 login.txt
 create mode 100644 registro.txt
```

Adicionalmente, sincronizamos los cambios con el repositorio remoto alojado en GitHub:

```
$ git push origin main
$ git push origin develop
```

Paso 7: Inspección Gráfica de Ramas en Terminal

Para verificar que el árbol de versiones refleje fielmente la topología de Gitflow, ejecutamos la instrucción de trazabilidad visual:

```
$ git log --graph --oneline --all
```

Resultados: Registro Visual del Grafo de Ramas

A continuación se presenta la salida textual exacta generada en la consola mediante 'git log --graph --oneline --all', la cual evidencia el nacimiento, bifurcación y convergencia de las ramas según los requerimientos solicitados para la entrega:

```

UCRT64: c:/Users/Quintanilla-Guzman/practica-gitflow

Quintanilla-Guzman@Quintanilla UCRT64 ~/practica-gitflow (main)
$ git push origin main --force
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 259 bytes | 259.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/realsivar/practica-gitflow.git
+ b86c589...09c9ccf main -> main (forced update)

Quintanilla-Guzman@Quintanilla UCRT64 ~/practica-gitflow (main)
$ git log --graph --oneline --all
* 09c9ccf (HEAD -> main, origin/main) Release: Publicación de versión a producción con login y registro
|
| * 8e2d9ee (origin/develop, develop) Merge branch 'feature/registro' into develop
| | \
| | * f9b15bc (feature/registro) feat(registro): Añade formulario de registro
| | /
| * 9ca4651 Merge branch 'feature/login' into develop
| | \
| | * 4cd41c5 (feature/login) fet(login): Añade formulario de login
| | /
| * f34549b Commit inicial del proyecto
$

```

Conclusiones

- **Dominio del flujo Gitflow:** Se logró implementar exitosamente el ciclo de vida de Gitflow a mano, dominando la creación de ramas principales y de soporte, el aislamiento de características y la resolución de fusiones.
- **Estabilidad del entorno de producción:** La rama 'main' se mantuvo estrictamente protegida de código en estado intermedio o inestable, asegurando que solo reciba cambios previamente validados en 'develop'.
- **Trazabilidad y auditoría:** El comando 'git log --graph --oneline --all' demostró ser una herramienta esencial de auditoría e inspección técnica para corroborar visualmente la estructura no lineal del historial del proyecto.
- **Preparación para CI/CD:** La ejecución manual de estos pasos sienta las bases conceptuales indispensables para la configuración de flujos automáticos mediante GitHub Actions y pipelines de CI/CD planificados para las sesiones 5 y 6.